

fast-jev-compaction 小白指南

把 Claude Code 的"压缩摘要"换成"决策式删除"——Jev 打分，只删不要的，留下的全部逐字

开源仓库：github.com/tamaratran/fast-jev-compaction (MIT, 7339★, TypeScript, v0.2.0)

🧠 决策式上下文压缩

🔗 Claude Code function-hook 插件

📦 npm 库 + 插件双形态

🚫 零 API 复刻实验 3 组全跑通

🔗 Jev 生态第二课 (继 jev-router)

这是什么？一句话说清

它是 Claude Code 的一个"记忆手术刀"插件：长会话快满时，不再让大模型写一份压缩摘要，而是让 Jev 决策模型对每一次工具调用逐个打分——**还有用的留下（一字不动），没用的删掉（连根拔），中间态截断。**

仓库双形态：**npm 库**（`npm install fast-jev-compaction`，任何 Agent 都能用）+ **Claude Code 插件**（function hooks 挂进 `session.compact`，把内置摘要整个替换掉）。作者是 tamaratran，2026-09-17 建仓，两天多 7339★。

一句话本质：上下文管理的思路从"改写历史"（summarize）换成了"只做减法"（delete）。减法不会错杀事实，因为留下的每个字都是原句。

为什么不用摘要？摘要错在哪

传统 compaction 让 LLM 把旧对话"总结成几段"。总结是有损压缩：**一个文件路径、一条精确报错、一句约束、一条命令，都可能在你还需要它的时候消失**。我在实验 2 里亲手复现了这个事故：

	传统摘要	fast-jev-compaction
用户约束 <code>Never edit src/generated</code>	✗ 丢了（下一轮模型就可能去改生成目录）	✓ 一字不动
约束 <code>CI token 在 .env.ci, 别打印</code>	✗ 丢了（泄密风险）	✓ 一字不动
工具结果原文（报错栈、行号）	被改写成"测试失败过"	有用则逐字保留，没用才删

有损摘要（改写历史）

"用户要求修测试.....改了个文件，"
"跑了测试，搜索了资料，修复了 TTL。"

路径没了 ✗ 约束没了 ✗
报错行号没了 ✗
错的内容还冒充"对的记忆"

决策式删除（只做减法）

约束原文 ✓ 原句逐字 ✓ 报错栈逐字 ✓
(Jev 判定"还需要"的那些)

没用的整条删掉 ✓
留下的不可能错 ✓

删错了也兜得住：工具随时能重跑

图 1：改写历史 vs 只做减法。减法的每一步都可审计（decisions 里有打分记录）。

"无损"到底无损在哪？它能动什么、不能动什么

用户和助手的文本消息永远逐字、永远保持原顺序，一个字都不会被改写或缩短。唯一被操作的对象是工具调用和工具结果这对儿（按 `tool_use_id` 配对）。三种处置：

处置	触发条件	结果
keep 全留	keepResult ≥ 0.5	调用+结果原文都留
drop_result 截留	keepResult<0.5 但 keepCall ≥ 0.5	结果截成前 300 字符 + 一行注记 <code>[truncated N chars; re-run the tool if needed]</code> ，调用本身留着
drop_call 整删	两个都 < 0.5	调用连同结果从历史里消失（消息内容全空就连消息一起删）

为什么删错也兜得住？因为**被删的东西本质都是"可以重跑再拿回来的东西"——文件能重读、测试能重跑、搜索能重搜**。这是整个设计的底气（README 原话：*the assistant can always re-run a tool or re-read a file*）。

Jev 被问了什么？"双问题"设计巧在哪

每个候选调用，Jev 收到**两个独立的 noul 问题**（noul 是 Jev 的题型：返回 0~1 的置信概率，不是生成文本）：

问题一 keepCall：知道"这次调用发生过、参数是什么"，对接下来还有价值吗？

→ 例子：Edit 过 src/generated — 文件内容没用，但"动过生成目录"必须记住

问题二 keepResult：这个调用的完整输出，还需要逐字留着吗？重跑一遍能不能代替？

→ 例子：测试失败的报错栈 — 重跑 npm test 拿不到当时的行号细节吗？能，但不必要

keepCall 高 + keepResult 低

"做过这件事"重要

"当时打印了什么"不重要

例：改过 src/generated ✎

→ 截留：留调用，结果截 300 字符

keepCall 低 + keepResult 低

做过什么都不重要

搜索了没用的资料、翻了没看的文件

例：一次空手的 WebSearch 🔍

→ 整删：连调用带结果一起消失

图 2：两个问题正交，组合出三种处置。实验 2 的 t4（Edit 生成文件）正是"做过≠内容有用"的活例子。

Jev 怎么"看"对话？state 的构造与 token 估算

发给 Jev 的 **state** = **完整对话史（从老到新）**，只是每个工具结果被换成一行短注记（`ok, 4213 chars (omitted)`），工具输入保留、文本保留、**什么都不总结**。再带上 **goal**（默认最近 3 条用户提问拼成任务描述）。

状态要塞进 **maxStateTokens**（默认 25k，Jev 请求上限 32k）。怎么知道多少 token？**不用分词器，纯字符启发式：英文单词每 6 个字母算 1 token，数字半个，其他符号 0.9 个**。我实测（实验 3）：

文本类型	估算 vs chars/4 粗算法	说明
中文	chars/4 少算 263%	每个汉字算 0.9 token——中文对话用 chars/4 会严重超限
JSON 重度 state	少算 51%	引号/花括号每个 0.9 token，README 声称最多差 40%，实测方向一致
英文散文	估高约 14%	刻意 宁高勿低 ：高估只是提前降级分批，低估会真撞 32k 上限

塞不下怎么办？8 档阶梯降级 + 保险丝

state 超预算时不一刀切，而是像下楼梯一样逐档降级，每档只在上一档不够用时才启用，能塞下就立刻停手（源的顺序是硬编码的——先动工具输入，再动文本，再压缩调用行）：



老消息优先降级，钉扎区（首条+最新 N 条）最后动、能不动就不动。每降一档立刻查：塞下了就停。

图 3：阶梯降级顺序（exp1 实测 7 档 + 保险丝触发）。

实验1实测（38 消息/17 调用合成会话）

预算 8000 tok → full (state 4367 tok)
预算 4000 tok → texts abridged(3989) ← 输入截断还不够，开始掐文本
预算 2500 tok → collapsed (2407) ← 旧消息整体变一行笔记
预算 1500 tok → old calls (1495) ← 旧调用缩成一行
预算 ≤1100 tok → throw: history too large for Jev ← 保险丝，宁可拒压不可瞎压
场景B(call-only 串) 1200 tok → merged (851) ← 相邻调用串合并复用信封

保险丝哲学：所有档位都试过还塞不下，就抛异常把决定权交还调用方（插件层回退到内置摘要）——宁可“这次没压成”，不可“压出一份错误记忆”。

几百个调用一次问得完吗？批处理设计

问题带着完整 state 发，state 本身可能已 25k token，所以问题必须分批：每批问题 + 整份 state ≤ maxRequestTokens（默认 30k）。关键设计：每一批都重发同一份完整 state——Jev 每次都看着全对话做判断，不会因为分批而“失忆”；各批并发执行，答案按问题名合并。

代价也写进了 Limitations：state 接近上限时，一批只装得下几个问题，一次压缩可能要发很多个请求（每个问题对约 80~100 token，30k-25k=5k 预算 ≈ 每批 50 个问题对）。

谁受特殊保护？钉扎与 goal

保护对象	规则
第一条消息	永远钉住——任务起点、原始指令都在这
最新 preserveRecentMessages 条 (默认 6)	永远钉住——刚发生的事 Jev 还没资格判"没用了"
无结果的调用	根本不进候选 (还没有可删的东西)
用户/助手文本	只能被"看", 不能被删改 (降级只发生在发给 Jev 的 state 里, 不动真历史)

实验 2 实证: t4 (Edit 生成文件) 落在最后 4 条钉扎区, Jev 就没被问过——`reason=pinned` 直接保留。同时验证了 **重建后不存在"孤儿结果" (result 没有配对的 call)**——那是 API 会直接报错的非法形态。

怎么挂进 Claude Code? function hooks 三分钟接入

① 前置开关 (function hooks 是 2.1.274+ 早期特性)

~/.claude/settings.json:

```
{ "env": { "CLAUDE_CODE_ENABLE_FUNCTION_HOOKS": "1",  
  "TYPESAFE_API_KEY": "<你的 key>" } }
```

② 装插件

```
claude plugin marketplace add tamaratran/fast-jev-compaction
```

```
claude plugin install fast-jev-compaction@fast-jev-compaction
```

③ 重启或 /reload-plugins → 之后 /compact 和自动压缩都走 Jev

toast: fast-jev-compaction: kept N/M messages, no summary (...)

回退时: fallback to built-in summary (...)

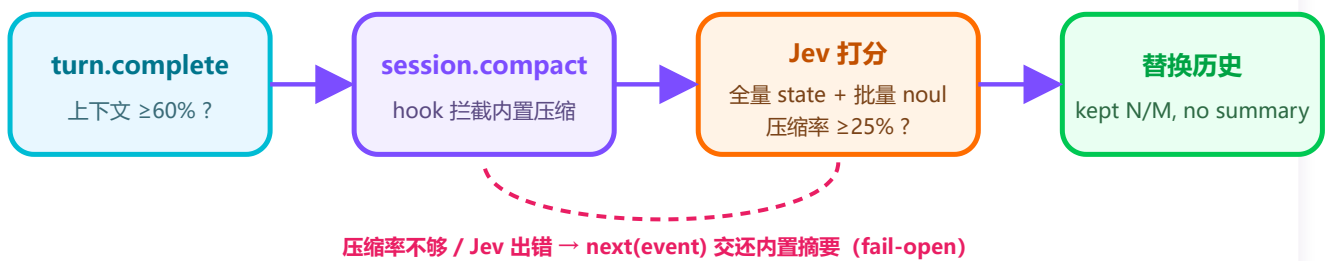


图 4: hook 双挂点。turn.complete 管自动触发, session.compact 管执行; 任何失败都回退内置摘要, 会话永不卡死。

什么时候宁可不动? "宁可不动"的三道闸

闸门	条件	动作
降压缩率闸	压完减少量 < minReductionRatio (默认 25%)	放弃成果, 回退内置摘要 (短会话压无可压)
异常闸	Jev 失败 / 答案畸形 / 没配 key / 塞不下 throw	catch 后回退内置摘要, toast 说明原因
严格校验闸	答案缺字段、noul 不是有限数字	直接 throw (noulAnswer 严格校验)

我踩的坑恰好证明第三道闸的价值: 复刻实验首跑时 askBatch 忘了从答案对象取 `.noul` 数值, t1 (明明该保留的调用) 被误判成整删——**畸形答案混进决策流, 比压缩失败更危险**。上游用严格校验 + throw + 回退把这类脏数据挡在历史之外。

它和 jev-router 什么关系？Jev 生态位全景

两个项目是 Jev（TypeSafe 的决策模型）生态里互补的两件套：

	jev-router（上期）	fast-jev-compact（本期）
解决的问题	省钱 ：每轮该用哪个档的模型	抗遗忘 ：长会话上下文满了怎么压
挂载点	回环代理劫持 ANTHROPIC_BASE_URL	function hooks 挂 session.compact
Jev 的角色	裁判：每轮选 Haiku/Sonnet/Opus	考官：对每个工具调用打 keep/drop 分
失败策略	fail-open 退安全档	fail-open 回退内置摘要
共同点	同一个 SystemOne 端点（ <code>/v1/systemone</code> ）；同样的“Jev 判断 + 兜底永不阻塞”哲学	

生态观察：README 里 `jev-latest` 模型、\$0.042/M 定价都与上期一致。这个插件 2 天 7339★ 说明“决策模型当上下文管家”是 Jev 落地最猛的用法——**不生成内容、只做判断，恰恰是决策模型的优势。**

我该用它吗？局限与冷启动建议

局限（README 自述 + 我的判断）	影响
只有工具调用/结果是候选	超长文本消息（贴了大日志的 user 消息）压不掉
token 是估算不是分词器	边界情况可能多分一批请求（浪费钱但不炸）
置信度 ≠ 安全证明	Jev 0.9 的 keep 也可能错删——兜底是“工具能重跑”
state 每批重发	接近 25k 上限的会话，一次压缩成本不低
隐私	整个对话史（含代码）要发给 TypeSafe 云端——与 jev-router 同一掂量

冷启动路径：① 无 key 也能玩：`npm install` 后用 `compact(messages, asker)` 注入自己的 mock asker（我实验 2 就是这么跑的，零 API）；② 有 key：插件安装即用，观察 toast 里的 `kept N/M` 和决策日志；③ 当库用：拿 `collectToolCalls / fitState / decideCall / applyDecisions` 这些导出积木自己搭管线。



这个项目给我什么（能力清单）

1. 给 Claude Code 装"无损记忆手术刀"——长任务不再越跑越失忆，约束、报错、路径永不蒸发；
2. 一套可移植的"决策式压缩"范式——任何 Agent（自己的 bot、其他 CLI）都能拿 `compactMessages` 库直接用；
3. Jev 双问题设计模式——"这件事发生过重要吗 / 它的产出还需要原文吗"可以平移到日志治理、缓存淘汰、聊天记录归档；
4. 阶梯降级 + 保险丝工程模板——"能塞就停、塞不下就 throw、throw 就回退"的三段式安全设计；
5. 零 API 教具——`JevAsker` 接缝 + mock asker，不花一分钱跑通全流程（我的复刻实验就是证明）；
6. 看懂 function hooks 新机制——Claude Code 2.1.274+ 的早期特性，`session.compact / turn.complete` 两个挂点。



动手实验（3 组全部真实运行，纯 Node 零依赖零 API）

环境说明 上游 `npm install` 被本机安全策略拦截，无法直接运行其测试套件；改为在 `exercise/replica.mjs` 里按上游源码逐函数忠实复刻（MIT 允许），驱动脚本全用 Node v24.21.0 裸跑。结论标注“复刻实证”。

实验 1：亲手拧 fitState 的“阶梯降级旋钮”（exp1_fit_state_ladder.mjs）

步骤 1 构造 38 条消息/17 次工具调用的合成会话（16 轮旧读写 + 3 条新消息）。

步骤 2 预算从 8000 一路收到 300，观察每档触发点。

目的 看清“能塞就停”的阶梯到底怎么走。✅ **预期/实测**：8000→`full(4367)`；4000→`texts abridged`；2500→`old messages collapsed`；1500→`old calls compacted`；≤1100→**保险丝 throw**；场景 B（call-only 串）1200→`old calls merged(851)`。8 档实证 7 档 + 保险丝。

实验 2：mock Jev 驱动完整压缩主流程（exp2_mock_jev_compact.mjs）

步骤 1 编排 4 次调用：Read 测试文件 / npm test / 空手 WebSearch / Edit 生成文件，mock asker 按语义剧本打分。

步骤 2 `node exp2_mock_jev_compact.mjs` 跑完整 `compact()`，检查 5 条不变量。

✅ **预期/实测**：t1→**kept 逐字保留**（含 `at auth.test.ts:42` 行号细节）；t2→**drop_result** 截 300 字符 + re-run 注记；t3→**drop_call** 整删；t4→**pinned**；孤儿结果 0；用户文本一字未动；压缩率 51.9%。对照传统摘要：“**Never edit src/generated**”和“**.env.ci token**”两约束全丢。

实验 3：token 估算器为什么不用 chars/4（exp3_estimator.mjs）

步骤 对中文/英文/JSON/代码行分别比较 `estimateTokens` 与 chars/4 粗算法。

✅ **预期/实测**：中文 chars/4 少算 **263%**（每汉字 0.9 tok）；JSON 重度 state 少算 **51%**（README 称最多 40%，方向一致）；英文散文反估高 14%——**宁高勿低**，高估只浪费一点预算，低估真撞 32k 上限。

源头：<https://x.com/KKaWSB/status/2101043826714661033>（KK.aWSB Jev 推荐帖）→ 开源插件溯源：

github.com/tamaratran/fast-jev-compaction（MIT, 7339★, v0.2.0, e3f262a）

姊妹篇：[jev-router 小白指南](#)（Jev 生态第一课：路由省钱）；知识库概念：决策式上下文压缩 / 双问题置信评估 / 阶梯降级与保险丝 / 无分词器 token 估算

产出日期 2026-10-03 · 全部实验真实运行于 Windows 11 + Node v24.21.0 · 复刻品声明见 `exercise/replica.mjs` 头注